CRACK YOUR INTERVIEW WITH EASE

Big Data Interview Prep Guide

Top 50+ Apache Spark Interview Questions and Answers

1. Best Spark Interview Questions and Answers

The **Big Data** technology is an umbrella term. It is emerging with time. **Apache Hadoop**, **Apache Spark** is the framework for dealing with this. The revenue of Big Data is increasing exponentially. To become a part of Bigdata industry I hope these Top 50+ Apache Spark Interview Questions and Answers will help to get an edge in Bigdata Market.

As a Bigdata professional, it is necessary to know the right buzzword, and the right answer to frequently asked Spark Interview questions and answers. The Apache Spark Interview Questions and answers mentioned below revolves around the concept of **Spark Core**, **Spark Streaming**, **Spark SQL**, **GraphX**, and **MLlib**.
Hope this blog will act as a gateway to your Spark Job.
So, let's explore top Spark Interview Questions and Answers.

2. Top Apache Spark Interview Questions and Answers

Here we are going to discuss the list of Spark interview questions and answers. We have classified these Apache Spark Interview Questions according to Spark ecosystem components-

a. Apache Spark Basic Interview Questions and Answers

Here are some Frequently asked Spark Interview Questions and Answers for freshers and experienced.

Q.1 What is Apache Spark?

Apache Spark is open source, wide range data processing engine. It is data processing engine with high APIs. It allows data worker to execute *streaming, machine learning or SQL* workloads. These jobs need fast iterative access to datasets. Spark provides API in various languages like **Python, R, Scala, Java**. We can run Spark by itself or on various existing cluster manager. There is various deployment option in Spark. For example, **Standalone Deploy Mode, Apache Mesos, Hadoop YARN.**
The design of the Apache is so dynamic that it can integrate with all the **Big Data** tools. For example, Spark can access data from any of the **Hadoop** data source. It can also run in Hadoop data cluster. Spark does not have its own storage system. It relies on **HDFS** or other file storage for storing the data.

Q.2 Why does the picture of Spark come into existence?
To overcome the drawbacks of **Apache Hadoop**, Spark came into the picture. Some of the drawbacks of Hadoop that Apache Spark overcomes are:

- Hadoop used only Java to build applications. Because it uses Java there were some security concerns as Java is prone to cyber crime.
- Apache Hadoop was apt only for batch processing. So, it does not support stream processing which was overcome in Spark.
- Hadoop used disk-based processing which results in slower retrieving of data. Spark overcomes this by in-memory computation.

Q.3 What are the features of Spark?
Some of the features of Apache Spark are:

- The processing speed of Apache Spark is very high.
- Apache Spark is dynamic in nature. There are about 80 high-level operators, thus, using which we can build a parallel application.
- We can reuse code for join stream against historical data or for batch processing.
- Through **RDD** we achieve fault tolerance. Thus, the data recovery is possible in RDD.

- Spark support many languages like **Java, Scala, Python, and R**. Thus, makes it more user-friendly and is dynamic in nature.
- It can run independently and also on other cluster managers like Hadoop YARN.
- Apache Spark is cost effective solution for Big data problem. While Hadoop needs large storage and the large data center during replication.

Q.4 What are the limitations of Spark?

- Does not have its file management system. Thus, it needs to integrate with Hadoop or other cloud-based data platforms.
- In-memory capability can become a bottleneck. Especially when it comes to cost-efficient processing of Bigdata.
- Memory consumption is very high. And the issues for the same are not handled in a user-friendly manner. d. It requires large data.
- MLlib lack in some available algorithms, for example, Tanimoto distance.

Q.5 List the languages supported by Apache Spark.

Apache Spark Supports the following Languages: **Scala, Java, R, Python.**

Q.6 What are the cases where Apache Spark surpasses Hadoop?

The data processing speed increases in the **Apache Spark**. This is because of the support of *in-memory computation* by the system. Thus, the performance of the system increase by 10x-1000x times. Apache Spark uses various languages for distributed application development.

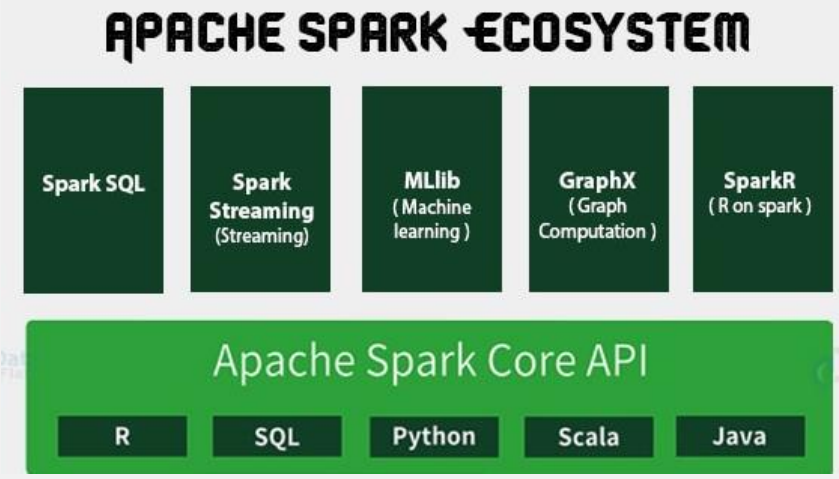
On the top of spark core, various libraries are present. These libraries enable workload that uses streaming, SQL, graph and machine learning. Some of these workloads are also supported by Hadoop. Spark facilitates the development by joining them into the same application. Apache Spark adopts micro-batching. Which is essentially used for handling near real time processing data model.

Q.7 Compare Hadoop and Spark.

- **Cost Efficient –** In Hadoop, during replication, a large number of servers, huge amount of storage, and the large data center is required. Thus, installing and using Apache Hadoop is expensive. While using Apache Spark is a cost effective solution for big data environment.

- **Performance –** The basic idea behind Spark was to improve the performance of data processing. And Spark did this to 10x-100x times. And all the credit of faster processing in Spark goes to in-memory processing of data. In Hadoop, the data processing takes place in disc while in Spark the data processing takes place in memory. It moves to the disc only when needed. The Spark in-memory computation is beneficial for iterative algorithms. When it comes to performance, because of batch processing in Hadoop it's processing is quite slow while the processing speed of Apache is faster as it supports micro-batching.
- **Ease of development –** The core in Spark is the distributed execution engine. Various languages are supported by Apache Spark for distributed application development. For example, Java, Scala, Python, and R. On the top of spark core, various libraries are built that enables workload. they make use of streaming, SQL, graph and machine learning. Hadoop also supports some of these workloads but Spark eases the development by combining all into the same application. d. Failure recovery: The method of Fault
- **Failure recovery –** The method of Fault Recovery is different in both Apache Hadoop and Apache Spark. In Hadoop after every operation data is written to disk. The data objects are stored in Spark in RDD distributed across data cluster. The RDDs are either in memory or on disk and provides full recovery from faults or failure.
- **File Management System –** Hadoop has its own File Management System called HDFS (Hadoop Distributed File System). While Apache Spark an integration with one, it may be even HDFS. Thus, Hadoop can run over Apache Spark.
- **Computation model –** Apache Hadoop uses batch processing model i.e. it takes a large amount of data and processes it. But Apache Spark adopts micro-batching. Must for handling near real time processing data model. When it comes to performance, because of batch processing in Hadoop it's processing is quite slow. The processing speed of Apache is faster as it supports micro-batching.

- **Lines of code –** Apache Hadoop has near about 23, 00,000 lines of code while Apache Spark has 20,000 lines of code.
- **Caching –** By caching partial result in memory of distributed workers Spark ensures low latency computations. While MapReduce is completely disk oriented, there is no provision of caching.
- **Scheduler** – Because of in-memory computation in Spark, it acts as its own flow scheduler. While with Hadoop MapReduce we need an extra job scheduler like Azkaban or Oozie so that we can schedule complex flows.
- **Spark API –** Because of very Strict API in Hadoop MapReduce, it is not versatile. But since Spark discards many low-level details it is more productive.
- **Window criteria –** Apache Spark has time-based window criteria. But Apache Hadoop does not have window criteria since it does not support streaming.
- **Faster –** Apache Hadoop executes job 10 to 100 times faster than Apache Hadoop MapReduce.
- **License –** Both Apache Hadoop and Apache MapReduce has a License Version 2.0.
- **DAG() –** In Apache Spark, there is cyclic data flow in machine learning algorithm, which is a direct acyclic graph. While in Hadoop MapReduce data flow does not have any loops, rather it is a chain of the image.
- **Memory Management –** Apache Spark has automatic memory management system. While Memory Management in Apache Hadoop can be either statistic or dynamic.
- **Iterative Processing –** In Apache Spark, the data iterates in batches. Here processing and scheduling of each iteration are separate. While in Apache Hadoop there is no provision for iterative processing.
- **Latency** – The time taken for processing by Apache Spark is less as compared to Hadoop since it caches its data on memory by means of RDD, thus the latency of Apache Spark is less as compared to Hadoop.



Spark Interview Questions and Answers – Spark Ecosystem

Q.8 What are the components of Spark Ecosystem?

The various components of Apache Spark are:

- **Spark Core –** Spark Core is the foundation of the whole project. All the functionality that is in Spark, is present on the top of Spark Core.
- **Spark Streaming –** It allows fault-tolerant streaming of live data streams. It is an add-on to core Spark API. Here it makes use of micro-batching for real-time streaming. Thus it packages live data into small batches and delivers to the batch system for processing.
- **Spark SQL –** Spark SQL component is distributed framework for structured data processing. Using Spark SQL Spark gets more information about the structure of data and the computation being performed. As a result, by using this information Spark can perform extra optimization.
- **Spark MLlib –** MLlib is a scalable learning library that discusses both: High-quality algorithm, High speed. The motive behind MLlib creation is to make machine learning scalable and easy. Thus. it contains machine learning libraries that have an implementation of various machine learning algorithms.
- **Spark GraphX** – GraphX is API for graphs and graph parallel execution. In order to support graph computation, graphX contains set of fundamental operators like sub graph, joinvertices and an optimized variant of Pregel API. Also, clustering,

classification, traversal, searching, and pathfinding is possible in graphX.

- **SparkR –** SparkR is Apache Spark 1.4 release. The key component of SparkR is SparkR DataFrame. *Data frames* are a fundamental data structure for **data processing in R** and the concept of data frames extends to other languages with libraries like Pandas etc.

Q.9 What is Spark Core?

Spark Core is a common execution engine for Spark platform. It provides *parallel* and *distributed processing* for large data sets. All the components on the top of it. Spark core provides speed through *in-memory computation*. And for ease of development, it also supports Java, Scala and Python APIs.

RDD is the basic data structure of Spark Core. RDDs are immutable, a partitioned collection of record that can operate in parallel. We can create RDDs by transformation on existing RDDs. Also by loading an external dataset from stable storage like HDFS or **HBase**, we can form RDD.

Q.10 How is data represented in Spark?

The data can be represented in three ways in Apache Spark: *RDD, DataFrame, DataSet*.

RDD: RDD stands for *Resilient Distributed Datasets*. It is also a *Read-only* partition collection of records. RDD is the fundamental data structure of Spark. Hence, RDDs can only be created through deterministic operation on either:

- Data in stable storage.
- Parallelizing already existing collection in driver program.
- Other RDDs. RDD allows programmer perform in-memory computations on large clusters in a fault-tolerant manner. Thus, speed up the task.

DataFrame: Unlike an RDD, the data organizes into named columns, like a table in a relational database. It is also an immutable distributed collection of data. **DataFrame** allows developers to impose a structure onto a distributed collection of data, therefore, allowing higher-level abstraction.

DataSet: Dataset is an extension of DataFrame API which provides type-safe, object-oriented programming interface. **DataSet** also takes advantage

of **Spark's Catalyst optimizer** by exposing expressions and data fields to a query planner.

Q.11 What are the abstractions of Apache Spark?

The main abstraction provided by Apache Spark is **Resilient Distributed Dataset**. RDDs are fault tolerant in nature. We cannot improve the changes made in RDD. RDDs creation starts with the file in a file system like Hadoop file system and then transforming it. The shared variable is the second abstraction provided by Apache Spark. We can use this in parallel operations.

Q.12 Explain the operations of Apache Spark RDD.

Apache Spark RDD supports two types of operations: *Transformations and Actions-*

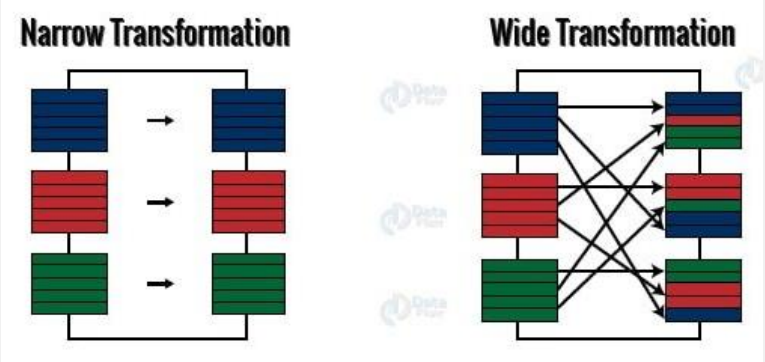
- **Transformations** are lazy operations on an RDD that create one or many new **RDDs**. For example *Map, filter, reduceByKey* etc creates new RDD. RDD create a new dataset from an existing one. That executes on demand. That means they compute lazily. Whenever we perform any transformation on RDD, it creates a new RDD each time.
- **Action** returns final result of RDD computations. It triggers execution using lineage graph to load the data into original RDD. After application of all the intermediate transformation, it gives the final result to *driver program* or writes it out to file system. Upon applying Actions on an RDD non-RDD values gets generate.

Q.13 How many types of Transformation are there?

There are two types of transformation namely *narrow transformation* and *wide transformation*.

- **Narrow Transformation** is the result of *map, filter* and such that the data is from a single partition only. As a result, the data is self-sustained. The RDD that we get as an output has a partition with records that originate from a single partition in the parent RDD.
- **Wide transformations** are the result of *groupByKey* and *reduceByKey.* The data that we will need to

compute the records in a single partition is kept at many partitions of the parent RDD.



Apache Spark Narrow and Wide Transformations

Q.14 In how many ways RDDs can be created? Explain.

There are three ways to create an RDD:

- **Parallelized collection –** In the initial stages, the RDD is generally created by parallelized collection. In this method, we take the existing collection in the program and pass it to *parallelize()* method of *SparkContext.* The thing that should be noticed in the parallelized collection is the number of partition the dataset is cut into. For each partition of the cluster, Spark will run one task. Spark set a number of partition based on our cluster. But the number of partitions can also be set manually. Pass the number of partition as the second parameter for manual partition. e.g. sc.parallelize(data, 20), here we have manually given a number of partition as 20.
- **External Datasets (Referencing a dataset) –** In Spark one can create distributed dataset from any data source supported by Hadoop. For example the local file system, HDFS, Cassandra, HBase etc. In this, the data is loaded from the external dataset. To create text file RDD we can use **SparkContext** textFile method. It takes URL of the file and read it as a collection of line. URL can be a local path on the machine or a hdfs://, s3n://, etc.
- **Creating RDD from existing RDD –** Transformation converts one RDD into another RDD. By using transformation we can create an RDD from existing RDD. Transformation acts as a function that intakes an RDD and produces one.

Q.15 What are Paired RDD?

Paired RDDs are the RDD-containing *key-value pair*. A key-value pair (KYP) contains two linked data item. Here *Key* is the identifier and *Value* are the data corresponding to the key value.

Q.16 What is meant by in-memory processing in Spark?

In **in-memory computation**, we keep data in random access memory in place of some slow disk drives. The processing of data is in parallel. Using this we can also identify the pattern, analyze large data Spark offers in in-memory capabilities. As a result, this increases the processing speed because it retrieves the data from memory in place of the disk. Also, the execution time of the process decreases. Keeping the data in-memory improves the performance by the order of magnitudes.

The main abstraction of Spark is its **RDDs**. Also, we can cache RDD using the ***cache()*** or ***persist()*** method. In *cache()* method all the RDD are in-memory. The dissimilarity between cache() and persist() is the default storage level. For cache() it is MEMORY_ONLY. While in *persist()* there are various storage levels like:

- MEMORY_ONLY,
- MEMORY_AND_DISK,
- MEMORY_ONLY_SER
- MEMORY_AND_DISK_SER
- DISK_ONLY

Q.17 How is fault tolerance achieved in Apache Spark?

The basic fault-tolerant semantic of Spark are:

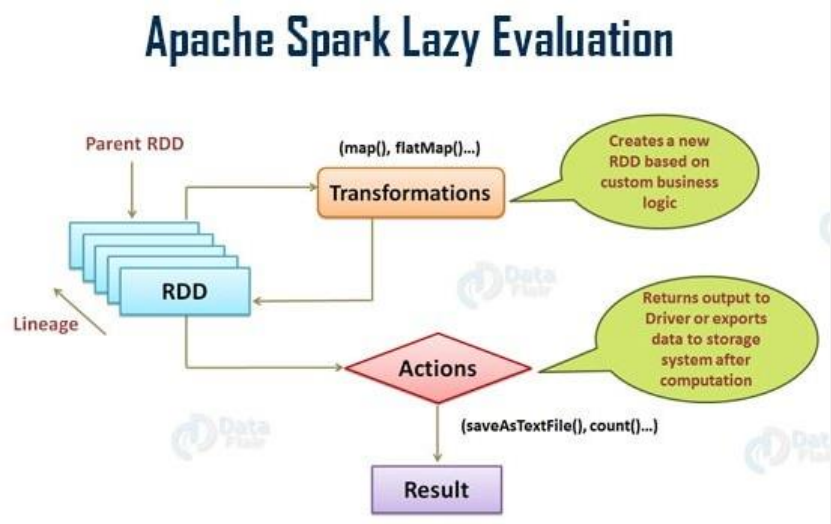
- Since all RDD is an immutable data set. Each RDD keeps track of the lineage of the deterministic operation that employee on fault-tolerant input dataset to create it.
- If any partition of an RDD is lost due to a worker node failure, then that partition can be re-computed from the original fault-tolerant dataset using the lineage of operations.
- Assuming that all of the RDD transformations are deterministic, the data in the final transformed RDD will always be the same irrespective of failures in the Spark cluster.

To achieve fault tolerance for all the generated RDDs, the achieved data replicates among multiple Spark executors in worker node in the cluster. This result in two types of data that should recover in the event of failure:

- **Data received and replicated –** In this, the data replicates on one of the other nodes. Thus we can retrieve data when a failure occurs.
- **Data received but buffered for replication –** the data does not replicate. Thus the only way to recover fault is by retrieving it again from the source.

Failure can also occur in worker and driver nodes.

- **Failure of worker node –** The node which runs the application code on the cluster is worker node. These are the slave nodes. Any of the worker nodes running executor can fail, thus resulting in loss of in-memory data. If any receivers were running on failed nodes, then their buffer data will vanish.
- **Failure of driver node –** If the driver node running the Spark Streaming application fails, then there is the loss of SparkContent. All executors along with their in-memory data vanishes.



Spark Interview Questions and Answers – Lazy Evaluation Feature

Q.18 What is Directed Acyclic Graph(DAG)?

RDDs are formed after every transformation. At high level when we apply action on these RDD, Spark creates a **DAG**. DAG is a finite directed graph with *no directed cycles*.
There are so many vertices and edges, where each edge is directed from one vertex to another. It contains a sequence of vertices such that every edge is directed from earlier to later in the sequence. It is a strict generalization of MapReduce model. DAG lets you get into the stage and expand in detail on any stage.
In the stage view, the details of all RDDs that belong to that stage are expanded.

Q.19 What is lineage graph?
Lineage graph refers to the graph that has all the parent RDDs of an RDD. It is the result of all the transformation on the RDD. It creates a logical execution plan.
A *logical execution plan* is a plan that starts with the very first RDD. Also, it does not have any dependency on any RDD. It then ends at the RDD which produces the result of an action that has been called to execute.
Q.20 What is lazy evaluation in Spark?
The **lazy evaluation** known as *call-by-need* is a strategy that delays the execution until one requires a value. The *transformation* in Spark is lazy in nature. Spark evaluate them lazily. When we call some operation in RDD it does not execute immediately; Spark maintains the graph of which operation it demands. We can execute the operation at any instance by calling the action on the data. The data does not loads until it is necessary.

Q.21 What are the benefits of lazy evaluation?
Using lazy evaluation we can:

- Increase the manageability of the program.
- Saves computation overhead and increases the speed of the system.
- Reduces the time and space complexity.
- provides the optimization by reducing the number of queries.

Q.22 What do you mean by Persistence?

RDD persistence is an optimization technique which saves the result of RDD evaluation. Using this we save the intermediate result for further use. It reduces the computation overhead. We can make persisted RDD through ***cache()*** and ***persist()*** methods. It is a key tool for the interactive algorithm. Because, when RDD is persisted each node stores any partition of it that it computes in memory. Thus makes it reusable for future use. This process speeds up the further computation ten times.

Q.23 Explain various level of persistence in Apache Spark.
The persist method allows seven storage level:

- **MEMORY_ONLY –** Store RDD as deserialized Java objects. If the RDD does not fit in memory, then some partitions will not be cached and will recompute on the fly each time needed. This is the default level.
- **MEMORY_AND_DISK –** Store RDD as deserialized Java objects. If the RDD does not fit in memory, store the partitions that don't fit on the disk, and read them from there when they're needed.
- **MEMORY_ONLY_SER (Java and Scala) –** Store RDD as serialized Java objects. This is more space-efficient than deserialized objects. especially when using a fast serializer. but it is hard for CPU to read.
- **MEMORY_AND_DISK_SER(Java and Scala) –** Like MEMORY_ONLY_SER, but spills partitions that don't fit in memory to disk.
- **DISK_ONLY –** It stores the RDD partitions only on disk.
- **MEMORY_ONLY_2, MEMORY_AND_DISK_2 –** It replicates each partition on two cluster nodes.
- **OFF_HEAP –** Like MEMORY_ONLY_SER, but store the data in off-heap memory. This requires enabling of off-heap memory.

Q.24 Explain the run-time architecture of Spark?
The components of the [run-time architecture of Spark]() are as follows:
a. The driver
b. Cluster manager
c. Executors

The Driver – The *main()* method of the program runs in the driver. The process that runs the user code which creates RDDs performs transformation and action, and also creates *SparkContext* is called diver. When the Spark Shell is launched, this signifies that we have created a driver program. The application finishes, as the driver terminates. Finally, driver program splits the Spark application into the task and schedules them to run on the executor.

Cluster Manager – Spark depends on cluster manager to launch executors. In some cases, even the drivers are launched by cluster manager. It is a pluggable component in Spark. On the cluster manager, the Spark scheduler schedules the jobs and action within a spark application in FIFO fashion. Alternatively, the scheduling can also be done in Round Robin fashion. The resources used by a Spark application can also be dynamically adjusted based on the workload. Thus, the application can free unused resources and request them again when there is a demand. This is available on all coarse-grained cluster managers, i.e. **standalone mode**, **YARN mode**, and **Mesos** coarse-grained mode.

The Executors – Each task in the Spark job runs in the *Spark executors*. thus, Executors are launched once in the beginning of Spark Application and then they run for the entire lifetime of an application. Even after the failure of Spark executor, the Spark application can continue with ease. There are two main roles of the executors:

- Runs the task that makes up the application and returns the result to the driver.
- Provide in-memory storage for RDDs that the user program cache.

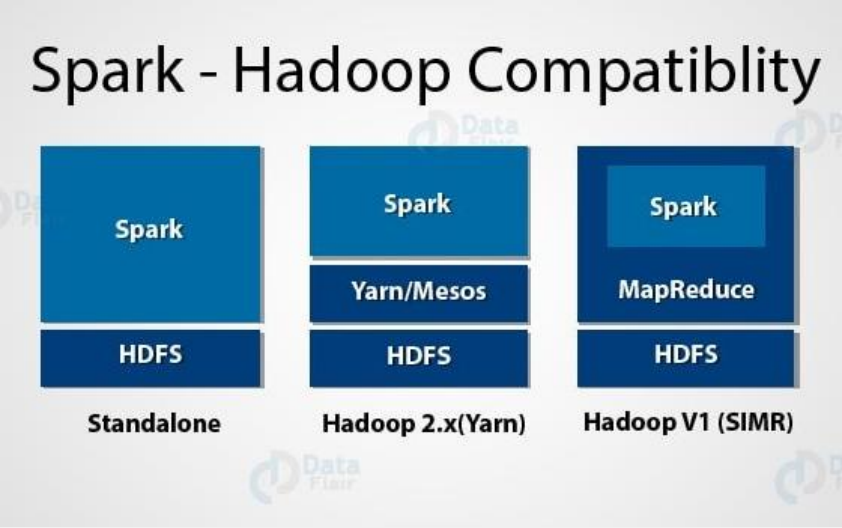
Q.25 Explain various cluster manager in Apache Spark?

The various cluster manages supported by Apache Spark are Standalone, Hadoop YARN, Apache Mesos.

- **Standalone Cluster Manager –** Standalone is a simple cluster manager of Spark that makes it easy to setup a cluster. In many cases, it is the simplest way to run Spark application in a clustered environment. It has masters and number of workers with the configured amount of memory and CPU cores. In standalone cluster mode, Spark allocates resources based on the

core. It has the constraints that only one executor can be allocated on each worker per application.

- **Hadoop YARN –** YARN became the sub-project of Hadoop in the year 2012. The key idea behind YARN is to bifurcate the functionality of resource manager and job scheduling into different daemons. The plan is to have a Global Resource Manager (RM) and per-application ***Application Master (AM)***. An application is either a **DAG** of graphs or an individual job. The data computation framework is a combination of the ResourceManager and the **NodeManager**.
- **Apache Mesos –** Apache Mesos handles the workload in a distributed environment.It is healthful for deployment and management of applications in large-scale cluster environments. Mesos clubs together the existing resource of the machines/nodes into a cluster, as a result from this a variety of workloads may utilize. This is known as node abstraction, thus it decreases an overhead of allocating a specific machine for different workloads. It is resource management platform for Hadoop and **Big Data** cluster. In some way, Apache Mesos is the reverse of virtualization. Because in virtualization one physical resource divides into multiple virtual resources, while in Mesos multiple physical resources groups into a single virtual resource.



Spark Interview Questions and Answers – Hadoop Compatibility

Read about Spark Cluster Managers in detail.

Q.26 In how many ways can we use Spark over Hadoop?

In three ways we can run Spark over Hadoop: Standalone, YARN, SIMR

- **Standalone –** In this, we can either divide resource on all machines or subset of machines in *Hadoop cluster.*
- **YARN –** We can run Spark on YARN without any pre-requisites. Thus, we can integrate Spark in Hadoop stack and take an advantage of facilities of Spark.
- **SIMR (Spark in MapReduce) –** Another way to do this is by launching Spark job inside Map reduce. With SIMR we can use Spark shell in few minutes after downloading it. This reduces the overhead of deployment, and we can play with Spark.

Q.27 What is YARN?

YARN became the sub-project of *Hadoop* in the year 2012. It is also known as MapReduce 2.0. The key idea behind YARN is to bifurcate the functionality of resource manager and job scheduling into different daemons. The plan is to have a Global *Resource Manager(RM)* and per-application *Application Master (AM)*. An application is either a **DAG** of graphs or an individual job.

The data computation framework is a combination of the ResourceManager and the NodeManager.

The **Resource Manager** manages resource among all the applications in the system. The Resource Manager has scheduled and *Application Manager.* The Scheduler allocates resource to the various running application. The Scheduler is pure Scheduler if it performs no monitoring or tracking of the status of the application. The **Application Manager** manages applications across all the nodes. NodeManager contains ApplicationMaster and container. A **container** is a place where a unit of work happens. Each task of MapReduce runs in one container. The per-application ApplicationMaster is a framework specific library. It negotiates resources from the ResourceManager and continues with **NodeManager(s)** to execute and watch the tasks. Application or job requires one or more containers. NodeManager looks after containers, resource usage (CPU, memory, disk, and network) and reporting this to the ResourceManager.

Q.28 How can we launch Spark application on YARN?

There are two deployment modes to launch Spark application on YARN: the *cluster mode* and the *client mode.*

- **In cluster mode,** the Spark driver runs inside Application Master Process and this is managed by YARN on the cluster.
- **In client mode,** the driver runs in the client process. The Application Master requests a resource from YARN. And it provides it to the driver program.

Q.29 Define Partition in Apache Spark.

Partition refers to, a logical block of large distributed Dataset. Logically partitioning the data and distributing it over the cluster provides parallelism. It also minimizes network traffic for sending data between executors. It determines how to access the entire hardware resources during job execution. RDD is automatically partitioned in Spark. We can change the size and number of the partition.

Q.30 What are shared variables?

Shared variables are one of the abstractions of Apache Spark. Shared variables can be used in parallel operations.

Whenever Spark runs a function in parallel as a set of tasks on different nodes, each variable that is used in function are circulated to each task. Sometimes there is a need to share the variables across the tasks or between the task and the driver program.

Apache Spark supports two types of shared variables namely ***broadcast variable*** and ***accumulator.***

Using broadcast variables we cache a value in memory on all nodes while we add accumulators to, such as counters and sums.

Q.31 What is Accumulator?

The *accumulator* is the type of Shared variable that is only added through associative and commutative operations. Using accumulator we can update the value of the variable while executing. We can also implement counters (as in MapReduce) or sums using an accumulator. Users can create named or unnamed accumulator. We can create numeric accumulator by calling *SparkContext.longAccumulator()* or *SparkContext.doubleAccumulator()* for Long or Double.

Q.32 What is the difference between DSM and RDD?

a) READ

- **RDD:** In RDD the read operation is *coarse grained or fine grained*. In **coarse-grained** we can transform the whole dataset but not an individual element. While in **fine-grained** we do the transformation of an individual element on a dataset.
- **Distributed Shared Memory:** The read operation in Distributed shared memory is fine-grained.

b) Write:

- **RDD:** The write operation is coarse-grained in RDD.
- **Distributed Shared Memory:** In distributed shared system the write operation is fine grained.

c) Consistency:

- **RDD:** The consistency of RDD is trivial meaning it is immutable in nature. Any changes made to an RDD cannot roll back, it is permanent. So the level of consistency is high.
- **Distributed Shared Memory:** The system guarantees that if the programmer follows the rules, the memory will be consistent. It also guarantees that the results of memory operations will be predictable.

d) Fault-recovery mechanism:

- **RDD:** Using lineage graph at any point in time we can easily find the lost data in an RDD.
- **Distributed Shared Memory:** Fault tolerance is achieved by a checkpointing technique. It allows applications to roll back to a recent checkpoint rather than restarting.

e) Straggler mitigation: Stragglers, in general, are those tasks that take more time to complete than their peers.

- **RDD:** in RDD it is possible to mitigate stragglers using backup task.
- **Distributed Shared Memory:** It is quite difficult to achieve straggler mitigation.

f) Behavior if not enough RAM:

- **RDD:** If there is not enough space to store RDD in RAM then the RDDs are shifted to disk.
- **Distributed Shared Memory:** In this type of system the performance decreases if the RAM runs out of storage.

Q.33 How can data transfer be minimized when working with Apache Spark?

By minimizing data transfer and avoiding shuffling of data we can increase the performance. In Apache Spark, we can minimize the data transfer in three ways:

- **By using a broadcast variable –** Since broadcast variable increases the efficiency of joins between small and large RDDs. the broadcast variable allows keeping a read-only variable cached on every machine in place of shipping a copy of it with tasks. We create broadcast variable v by calling *SparlContext.broadcast(v)* and we can access its value by calling the value method.
- **Using Accumulator –** Using accumulator we can update the value of a variable in parallel while executing. Accumulators can only be added through the associative and commutative operation. We can also implement counters (as in MapReduce) or sums using an accumulator. Users can create named or unnamed accumulator. We can create numeric accumulator by calling *SparkContext.longAccumulator()* or *SparkContext.doubleAccumulator()* for Long or Double respectively.
- By avoiding operations like ByKey, repartition or any other operation that trigger shuffle. we can minimize the data transfer.

Q.34 How does Apache Spark handles accumulated Metadata?

By triggering automatic cleanup Spark handles the automatic Metadata. We can trigger cleanup by setting the parameter "***spark.cleaner.ttl***". the default value for this is infinite. It tells for how much duration Spark will remember the metadata. It is periodic cleaner. And also ensure that metadata older than the set duration will vanish. Thus, with its help, we can run Spark for many hours.

Q.35 What are the common faults of the developer while using Apache Spark?

The common mistake by developers are:

- Customer hit web-service several time by using multiple clusters.
- Customer runs everything on local node instead of distributing it.

Q.36 Which among the two is preferable for the project- Hadoop MapReduce or Apache Spark?
The answer to this question depends on the type of project one has. As we all know Spark makes use of a large amount of RAM and also needs a dedicated machine to provide an effective result. Thus the answer depends on the project and the budget of the organization.

Q.37 List the popular use cases of Apache Spark.
The most popular use-cases of Apache Spark are:
1. Streaming
2. Machine Learning
3. interactive Analysis
4. fog computing
5. Using Spark in the real world

Q.38 What is Spark.executor.memory in a Spark Application?
The default value for this is 1 GB. It refers to the amount of memory that will be used per executor process.
We have categorized the above Spark Interview Questions and Answers for Freshers and Experienced-

- Spark Interview Questions and Answers for Fresher – Q.No.1-8, 37
- Spark Interview Questions and Answers for Experienced – Q.No. 9-36, 38

b. Spark SQL Interview Questions and Answers

In this section, we will discuss some basic Spark SQL Interview Questions and Answers.

Q.39 What is DataFrames?
It is a collection of data which organize in named columns. It is theoretically equivalent to a table in relational database. But it is more optimized. Just like RDD, DataFrames evaluates lazily. Using lazy evaluation we can optimize the execution. It optimizes by applying the techniques such as bytecode generation and predicate push-downs.

Q.40 What are the advantages of DataFrame?

1. It makes large data set processing even easier. Data Frame also allows developers to impose a structure onto a distributed collection of data. As a result, it allows higher-level abstraction.
2. Data frame is both space and performance efficient.
3. It can deal with both structured and unstructured data formats, for example, Avro, CSV etc . And also storage systems like HDFS, HIVE tables, MySQL, etc.
4. The DataFrame API's are available in various programming languages. For example Java, Scala, Python, and R.
5. It provides Hive compatibility. As a result, we can run unmodified Hive queries on existing Hive warehouse.
6. Catalyst tree transformation uses DataFrame in four phases: a) Analyze logical plan to solve references. b) Logical plan optimization c) Physical planning d) Code generation to compile part of the query to Java bytecode.
7. It can scale from kilobytes of data on the single laptop to petabytes of data on the large cluster.

Q.41 What is DataSet?

Spark Datasets are the extension of Dataframe API. It creates object-oriented programming interface and type-safety. Dataset is Spark 1.6 release. It makes use of Spark's catalyst optimizer. It reveals expressions and data fields to a query optimizer. Dataset also influences fast in-memory encoding. It also provides provision for compile time type-safety. We can check for errors in an application when they run.

Q.42 What are the advantages of DataSets?

- It provides run-time type safety.
- Influences fast in-memory encoding.
- It provides a custom view of structured and semi-structured data.
- It owns rich semantics and an easy set of domain-specific operations, as a result, it facilitates the use of structured data.
- Dataset API decreases the use of memory. As Spark knows the structure of data in the dataset, thus it creates an optimal layout in memory while caching.

Q.43 Explain Catalyst framework.

The **Catalyst** is a framework which represents and manipulate a DataFrame graph. Data flow graph is a tree of relational operator and expressions. The three main features of catalyst are:

- It has a TreeNode library for transforming tree. They are expressed as Scala case classes.
- A logical plan representation for relational operator.
- Expression library.

The *TreeNode* builds a query optimizer. It contains a number of the query optimizer. **Catalyst Optimizer** supports both *rule-based* and *cost-based optimization*. In rule-based optimization the optimizer use set of rule to determine how to execute the query. While the cost based optimization finds the most suitable way to carry out SQL statement. In cost-based optimization, many plans are generates using rules. And after this, it computes their cost. Catalyst optimizer makes use of standard features of Scala programming like pattern matching.

Q.44 List the advantage of Parquet files.

- It is efficient for large scale queries.
- It supports various efficient compression and encoding Scheme.
- It consumes less space.

We have categorized the above frequently asked Spark Interview Questions and Answers for Freshers and Experienced-

- **Spark Interview Questions and Answers for Fresher – Q.No. 39-42**
- **Spark Interview Questions and Answers for Experienced – Q.No. 43, 44**

c. Spark Streaming Interview Questions and Answers

In this section, we will discuss some basic Spark Interview Questions and Answers based on Spark Streaming.

Q.45 What is Spark Streaming?

Through ***Spark streaming,*** we achieve fault tolerant processing of live data stream. The input data can be from any source. For example, like *Kafka,*

Flume, kinesis, twitter or HDFS/S3. It gives the data to filesystems, databases, and live dashboards after processing. The working of Spark Streaming is as under:

- Spark Streaming takes in the live data.
- The input data stream is divided into batches of input data.
- The Spark engine processes the batches of input data. The final result is also in batches.

Q.46 What is DStream?

DStream is the high-level abstraction provided by *Spark Streaming*. It represents a continuous stream of data. Thus, DStream is internally a sequence of RDDs. There are two ways to create DStream:

- by using data from different sources such as Kafka, Flume, and Kinesis.
- by applying high-level operations on other DStreams.

Q.47 Explain different transformation on DStream.

DStream is a basic abstraction of Spark Streaming. It is a continuous sequence of RDD which represents a continuous stream of data. Like RDD, DStream also supports many transformations which are available on normal Spark RDD. For example*, map(func), flatMap(func), filter(func)* etc.

Q.48 Does Apache Spark provide checkpointing?

Yes, Apache Spark provides checkpointing. Apache supports two types of checkpointing:

- **Reliable Checkpointing:** It refers to that checkpointing in which the actual RDD is saved in the reliable distributed file system, e.g. HDFS. To set the checkpoint directory call: *SparkContext.setCheckpointDir(directory: String)*. When running on the cluster the directory must be an HDFS path since the driver tries to recover the checkpointed RDD from a local file. While the checkpoint files are actually on the executor's machines.
- **Local Checkpointing:** In this checkpointing, in Spark Streaming or GraphX we truncate the RDD lineage graph in Spark. In this, the RDD is persisted to local storage in the executor.

Q.49 What is write ahead log(journaling)?

The ***write-ahead log*** is a technique that provides durability in a database system. It works in the way that all the operation that applies on data, we write it to write-ahead log. The logs are durable in nature. Thus, when the failure occurs we can easily recover the data from these logs. When we enable the write-ahead log Spark stores the data in fault-tolerant file system.

Q.50 What is a reliable and unreliable receiver in Spark?

- **Reliable Receiver –** A reliable receiver acknowledges to the source on receiving data and stores it. Implementing this receiver involves examining of the semantics of source acknowledgments.
- **Unreliable Receiver –** An unreliable receiver does not send an acknowledgment to a source. It is for sources that do not bear an acknowledgment. It is also for reliable sources when one does not want to enter the complexity of acknowledgment.

We have categorized the above Spark Interview Questions and Answers for Freshers and Experienced-

- **Spark Interview Questions and Answers for Fresher – Q.No. 45-47**
- **Spark Interview Questions and Answers for Experienced – Q.No. 48-50**

d. Spark MLlib Interview Questions and Answers

Here are some Spark Interview Questions and Answers for freshers and experienced based on MLlib.

Q.51 What is Spark MLlib?

MLlib is the name of *Spark's machine learning library*. The various tools provided by MLlib are:

- **ML Algorithms:** It contains common learning algorithms such as classification, regression, etc.
- **Featurization:** it has tools for feature extraction, dimensionality reduction, and selection. It also has tools for constructing, evaluating and tuning ML Pipelines.

Q.52 What is Sparse Vector?

A local vector contains both the integer type and 0-based indices. It also has double-typed values, which is stored on a single machine. In ***MLlib*** two types of local vectors are a supportive namely Dense and Sparse vector. The sparse vector is one in which most of the entries are zero.

Q.53 How to create a Sparse vector from a dense vector?

Vector sparseVector = Vectors.sparse(4, new int[] {1, 3}, new double[] {3.0, 4.0});

APACHE SPARK INTERVIEW QUESTIONS

ABHINANDAN PATRA

DATA ENGINEER

1. What is Apache Spark?

Apache Spark is an open-source, distributed computing system that provides an interface for programming entire clusters with implicit data parallelism and fault tolerance. It is designed to process large-scale data efficiently.

2. Why Apache Spark?

Apache Spark is used because it is faster than traditional big data tools like Hadoop MapReduce due to its in-memory processing capabilities, supports multiple languages (Scala, Python, R, Java), provides libraries for various tasks (SQL, machine learning, graph processing, etc.), and has robust fault tolerance.

3. What are the components of the Apache Spark Ecosystem?

The main components are:

- **Spark Core**: The foundational engine for large-scale parallel and distributed data processing.
- **Spark SQL**: For structured data processing.
- **Spark Streaming**: For real-time data processing.
- **MLlib**: A library for scalable machine learning.
- **GraphX**: For graph and graph-parallel computation.

4. What is Spark Core?

Spark Core is the general execution engine for the Spark platform, responsible for tasks such as scheduling, distributing, and monitoring applications.

5. Which languages does Apache Spark support?

Apache Spark supports:

- Scala
- Python
- Java
- R
- SQL

6. How is Apache Spark better than Hadoop?

Spark is better in several ways, including faster processing due to in-memory computation, ease of use with APIs for various programming languages, flexibility with built-in libraries for diverse tasks, and a rich set of APIs for transformations and actions.

7. What are the different methods to run Spark over Apache Hadoop?

Spark can run on Hadoop in the following modes:

- **Standalone**: Using its cluster manager.
- **YARN**: Hadoop's cluster manager.
- **Mesos**: Another cluster manager.

8. What is SparkContext in Apache Spark?

`SparkContext` is the entry point for any Spark application. It acts as a connection to the Spark cluster, allowing Spark jobs to be executed.

9. What is SparkSession in Apache Spark?

`SparkSession` is the unified entry point to work with DataFrames, Datasets, and SQL in Apache Spark. It replaces `SQLContext` and `HiveContext`.

10. SparkSession vs. SparkContext in Apache Spark

`SparkSession` is a combination of `SQLContext`, `HiveContext`, and `SparkContext` to provide a single point of entry to interact with Spark.

11. What are the abstractions of Apache Spark?

The primary abstractions are:

- **RDD (Resilient Distributed Dataset)**
- **DataFrames**
- **Datasets**

12. How can we create RDD in Apache Spark?

RDDs can be created in three ways:

- **Parallelizing a collection** in your program.
- **Referencing a dataset** in an external storage system (e.g., HDFS, S3, etc.).
- **Transforming** an existing RDD.

13. Why is Spark RDD immutable?

RDDs are immutable to provide fault tolerance and support functional programming principles, allowing Spark to rebuild lost data from the lineage information.

14. Explain the term paired RDD in Apache Spark.

Paired RDDs are RDDs where each element is a pair (key-value). They are used for operations like aggregation, grouping, and joins.

15. How is RDD in Spark different from Distributed Storage Management?

RDD is an in-memory data structure optimized for processing, while Distributed Storage (like HDFS) focuses on data storage and retrieval.

16. Explain transformation and action in RDD in Apache Spark.

- **Transformation**: Lazy operations that define a new RDD without executing until an action is called (e.g., `map`, `filter`).
- **Action**: Triggers the execution of transformations (e.g., `count`, `collect`).

17. What are the types of Apache Spark transformations?

Transformations can be narrow (e.g., `map`, `filter`) or wide (e.g., `groupByKey`, `reduceByKey`).

18. Explain the RDD properties.

RDD properties include:

- **Immutability**: Once created, RDDs cannot be changed.
- **Partitioned**: Distributed across various nodes in the cluster.
- **Lazy evaluation**: Operations are computed when an action is called.
- **Fault tolerance**: Recomputed using lineage information.

19. What is a lineage graph in Apache Spark?

A lineage graph tracks the sequence of transformations that created an RDD, used for recomputing lost data due to node failures.

20. Explain the terms Spark Partitions and Partitioners.

- **Partitions**: Logical division of data in RDDs, physically stored across nodes.
- **Partitioner**: Determines how data is distributed across partitions (e.g., `HashPartitioner`, `RangePartitioner`).

21. By default, how many partitions are created in RDD in Apache Spark?

By default, Spark creates partitions based on the number of cores available or the input file's HDFS block size.

22. What is Spark DataFrames?

DataFrames are distributed collections of data organized into named columns, similar to tables in a relational database.

23. What are the benefits of DataFrame in Spark?

Benefits include optimizations (Catalyst query optimizer), improved performance, and easier manipulation using SQL-like syntax.

24. What is Spark Dataset?

A Dataset is a distributed collection of data that provides type safety and object-oriented programming interfaces.

25. What are the advantages of datasets in Spark?

Advantages include compile-time type safety, optimizations through Tungsten, and the ability to leverage JVM object serialization.

26. What is Directed Acyclic Graph (DAG) in Apache Spark?

A DAG in Spark represents a sequence of computations performed on data, where each node is an RDD and edges represent transformations. It's used to optimize execution plans.

27. What is the need for Spark DAG?

The DAG allows Spark to optimize execution by scheduling tasks efficiently, minimizing data shuffling, and managing dependencies.

28. What is the difference between DAG and Lineage?

- **DAG**: Represents the entire execution plan of a Spark application.
- **Lineage**: Tracks transformations on a particular RDD, useful for fault recovery.

29. What is the difference between Caching and Persistence in Apache Spark?

- **Caching**: Default storage level is in-memory (`MEMORY_ONLY`).
- **Persistence**: Allows choosing different storage levels (disk, memory, etc.) for storing RDDs.

30. What are the limitations of Apache Spark?

Limitations include high memory consumption, limited built-in libraries compared to Hadoop, not suitable for small data or real-time streaming without specific tools.

31. Different Running Modes of Apache Spark

Spark can run in:

- **Local mode**: Single machine.
- **Standalone mode**: Using its cluster manager.
- **YARN mode**: On Hadoop’s cluster manager.
- **Mesos mode**: On Mesos cluster manager.
- **Kubernetes mode**: On Kubernetes.

32. What are the different ways of representing data in Spark?

Data can be represented as:

- **RDDs (Resilient Distributed Datasets)**

- **DataFrames**
- **Datasets**

33. What is Write-Ahead Log (WAL) in Spark?

Write-Ahead Log is a fault-tolerance mechanism where every received data is first written to a log file (disk) before processing, ensuring no data loss.

34. Explain Catalyst Query Optimizer in Apache Spark.

Catalyst is Spark SQL's query optimizer that uses rule-based and cost-based optimization techniques to generate efficient execution plans.

35. What are shared variables in Apache Spark?

Shared variables are variables that can be used by tasks running on different nodes:

- **Broadcast variables**: Efficiently share read-only data across nodes.
- **Accumulators**: Used for aggregating information (e.g., sums) across tasks.

36. How does Apache Spark handle accumulated metadata?

Spark stores metadata like lineage information, partition data, and task details in the driver and worker nodes, managing it using its DAG scheduler.

37. What is Apache Spark's Machine Learning Library?

MLlib is Spark's scalable machine learning library, which provides algorithms and utilities for classification, regression, clustering, collaborative filtering, and more.

38. List commonly used Machine Learning Algorithms.

Common algorithms in Spark MLlib include:

- Linear Regression
- Logistic Regression
- Decision Trees
- Random Forests
- Gradient-Boosted Trees
- K-Means Clustering

39. What is the difference between DSM and RDD?

- **DSM (Distributed Storage Management)**: Focuses on data storage across clusters.
- **RDD (Resilient Distributed Dataset)**: Focuses on distributed data processing with fault tolerance.

40. List the advantage of Parquet file in Apache Spark.

Advantages of Parquet files:

- Columnar storage format, optimized for read-heavy workloads.
- Efficient compression and encoding schemes.
- Schema evolution support.

41. What is lazy evaluation in Spark?

Lazy evaluation defers execution until an action is performed, optimizing the execution plan by reducing redundant computations.

42. What are the benefits of Spark lazy evaluation?

Benefits include:

- Reducing the number of passes over data.
- Optimizing the computation process.
- Decreasing execution time.

43. How much faster is Apache Spark than Hadoop?

Apache Spark is generally up to 100x faster than Hadoop for in-memory processing and up to 10x faster for on-disk data.

44. What are the ways to launch Apache Spark over YARN?

Spark can be launched over YARN in:

- **Client mode**: Driver runs on the client machine.
- **Cluster mode**: Driver runs inside YARN cluster.

45. Explain various cluster managers in Apache Spark.

Spark supports:

- **Standalone Cluster Manager**: Default cluster manager.
- **Apache Mesos**: A general-purpose cluster manager.
- **Hadoop YARN**: A resource manager for Hadoop clusters.
- **Kubernetes**: For container orchestration.

46. What is Speculative Execution in Apache Spark?

Speculative execution is a mechanism to detect slow-running tasks and run duplicates on other nodes to speed up the process.

47. How can data transfer be minimized when working with Apache Spark?

Data transfer can be minimized by:

- Reducing shuffling and repartitioning.
- Using broadcast variables.
- Efficient data partitioning.

48. What are the cases where Apache Spark surpasses Hadoop?

Apache Spark outperforms Hadoop in scenarios involving iterative algorithms, in-memory computations, real-time analytics, and complex data processing workflows.

49. What is an action, and how does it process data in Apache Spark?

An action is an operation that triggers the execution of transformations (e.g., `count`, `collect`), performing computations and returning a result.

50. How is fault tolerance achieved in Apache Spark?

Fault tolerance is achieved through lineage information, allowing RDDs to be recomputed from scratch if a partition is lost.

51. What is the role of the Spark Driver in Spark applications?

The Spark Driver is responsible for converting the user's code into tasks, scheduling them on executors, and collecting the results.

52. What is a worker node in an Apache Spark cluster?

A worker node is a machine in a Spark cluster where the actual data processing tasks are executed.

53. Why is Transformation lazy in Spark?

Transformations are lazy to build an optimized execution plan (DAG) and to avoid unnecessary computation.

54. Can I run Apache Spark without Hadoop?

Yes, Spark can run independently using its built-in cluster manager or other managers like Mesos and Kubernetes.

55. Explain Accumulator in Spark.

An accumulator is a variable used for aggregating information across executors, like counters in MapReduce.

56. What is the role of the Driver program in a Spark Application?

The Driver program coordinates the execution of tasks, maintains the SparkContext, and communicates with the cluster manager.

57. How to identify that a given operation is a Transformation or Action in your program?

Transformations return RDDs (e.g., `map`, `filter`), while actions return non-RDD values (e.g., `collect`, `count`).

58. Name the two types of shared variables available in Apache Spark.

- **Broadcast Variables**
- **Accumulators**

59. What are the common faults of developers while using Apache Spark?

Common faults include:

- Inefficient data partitioning.
- Excessive shuffling and data movement.
- Inappropriate use of transformations and actions.
- Not leveraging caching and persistence properly.

60. By Default, how many partitions are created in RDD in Apache Spark?

The default number of partitions is based on the number of cores available in the cluster or the HDFS block size.

61. Why do we need compression, and what are the different compression formats supported?

Compression reduces the storage size of data and speeds up data transfer. Spark supports several compression formats:

- **Snappy**
- **Gzip**
- **Bzip2**
- **LZ4**
- **Zstandard (Zstd)**

62. Explain the filter transformation.

The `filter` transformation creates a new RDD by selecting only elements that satisfy a given predicate function.

63. How to start and stop Spark in the interactive shell?

To start Spark in the interactive shell:

- Use `spark-shell` for Scala or `pyspark` for Python. To stop Spark:
- Use `:quit` or `Ctrl + D` in the shell.

64. Explain the `sortByKey()` operation.

`sortByKey()` sorts an RDD of key-value pairs by the key in ascending or descending order.

65. Explain `distinct()`, `union()`, `intersection()`, and `subtract()` transformations in Spark.

- **distinct()**: Returns an RDD with duplicate elements removed.
- **union()**: Combines two RDDs into one.
- **intersection()**: Returns an RDD with elements common to both RDDs.
- **subtract()**: Returns an RDD with elements in one RDD but not in another.

66. Explain `foreach()` operation in Apache Spark.

`foreach()` applies a function to each element in the RDD, typically used for side effects like updating an external data store.

67. `groupByKey` vs `reduceByKey` in Apache Spark.

- **groupByKey**: Groups values by key and shuffles all data across the network, which can be less efficient.
- **reduceByKey**: Combines values for each key locally before shuffling, reducing network traffic.

68. Explain `mapPartitions()` and `mapPartitionsWithIndex()`.

- **mapPartitions()**: Applies a function to each partition of the RDD.
- **mapPartitionsWithIndex()**: Applies a function to each partition, providing the partition index.

69. What is `map` in Apache Spark?

`map` is a transformation that applies a function to each element in the RDD, resulting in a new RDD.

70. What is `flatMap` in Apache Spark?

`flatMap` is a transformation that applies a function to each element, resulting in multiple elements (a flat structure) for each input.

71. Explain `fold()` operation in Spark.

`fold()` aggregates the elements of an RDD using an associative function and a "zero value" (an initial value).

72. Explain `createOrReplaceTempView()` API.

`createOrReplaceTempView()` registers a DataFrame as a temporary table in Spark SQL, allowing it to be queried using SQL.

73. Explain `values()` operation in Apache Spark.

`values()` returns an RDD containing only the values of key-value pairs.

74. Explain `keys()` operation in Apache Spark.

`keys()` returns an RDD containing only the keys of key-value pairs.

75. Explain `textFile` vs `wholeTextFiles` in Spark.

- **textFile()**: Reads a text file and creates an RDD of strings, each representing a line.
- **wholeTextFiles()**: Reads entire files and creates an RDD of (filename, content) pairs.

76. Explain `cogroup()` operation in Spark.

`cogroup()` groups data from two or more RDDs sharing the same key.

77. Explain `pipe()` operation in Apache Spark.

`pipe()` passes each partition of an RDD to an external script or program and returns the output as an RDD.

78. Explain Spark `coalesce()` operation.

`coalesce()` reduces the number of partitions in an RDD, useful for minimizing shuffling when reducing the data size.

79. Explain the `repartition()` operation in Spark.

`repartition()` reshuffles data across partitions, increasing or decreasing the number of partitions, involving a full shuffle of data.

80. Explain `fullOuterJoin()` operation in Apache Spark.

`fullOuterJoin()` returns an RDD with all pairs of elements for matching keys and `null` for non-matching keys from both RDDs.

81. Explain Spark `leftOuterJoin()` and `rightOuterJoin()` operations.

- **leftOuterJoin()**: Returns all key-value pairs from the left RDD and matching pairs from the right, filling with `null` where no match is found.
- **rightOuterJoin()**: Returns all key-value pairs from the right RDD and matching pairs from the left, filling with `null` where no match is found.

82. Explain Spark `join()` operation.

`join()` returns an RDD with all pairs of elements with matching keys from both RDDs.

83. Explain `top()` and `takeOrdered()` operations.

- **top()**: Returns the top `n` elements from an RDD in descending order.
- **takeOrdered()**: Returns the top `n` elements from an RDD in ascending order.

84. Explain `first()` operation in Spark.

`first()` returns the first element of an RDD.

85. Explain `sum()`, `max()`, `min()` operations in Apache Spark.

These operations compute the sum, maximum, and minimum of elements in an RDD, respectively.

86. Explain `countByValue()` operation in Apache Spark RDD.

`countByValue()` returns a map of the counts of each unique value in the RDD.

87. Explain the `lookup()` operation in Spark.

`lookup()` returns the list of values associated with a given key in a paired RDD.

88. Explain Spark `countByKey()` operation.

`countByKey()` returns a map of the counts of each key in a paired RDD.

89. Explain Spark `saveAsTextFile()` operation.

`saveAsTextFile()` saves the RDD content as a text file or set of text files.

90. Explain `reduceByKey()` Spark operation.

`reduceByKey()` applies a reducing function to the elements with the same key, reducing them to a single element per key.

91. Explain the operation `reduce()` in Spark.

`reduce()` aggregates the elements of an RDD using an associative and commutative function.

92. Explain the action `count()` in Spark RDD.

`count()` returns the number of elements in an RDD.

93. Explain Spark `map()` transformation.

`map()` applies a function to each element of an RDD, creating a new RDD with the results.

94. Explain the `flatMap()` transformation in Apache Spark.

`flatMap()` applies a function that returns an iterable to each element and flattens the results into a single RDD.

95. What are the limitations of Apache Spark?

Limitations include high memory consumption, not ideal for OLTP (transactional processing), lack of a mature security framework, and dependency on cluster resources.

96. What is Spark SQL?

Spark SQL is a Spark module for structured data processing, providing a DataFrame API and allowing SQL queries to be executed.

97. Explain Spark SQL caching and uncaching.

- **Caching**: Storing DataFrames in memory for faster access.
- **Uncaching**: Removing cached DataFrames to free memory.

98. Explain Spark Streaming.

Spark Streaming is an extension of Spark for processing real-time data streams.

99. What is DStream in Apache Spark Streaming?

DStream (Discretized Stream) is a sequence of RDDs representing a continuous stream of data.

100. Explain different transformations in DStream in Apache Spark Streaming.

Transformations include:

- **map()**, **flatMap()**, **filter()**
- **reduceByKeyAndWindow()**
- **window()**, **countByWindow()**
- **updateStateByKey()**

101. What is the Starvation scenario in Spark Streaming?

Starvation occurs when all tasks are waiting for resources that are occupied by other long-running tasks, leading to delays or deadlocks.

102. Explain the level of parallelism in Spark Streaming.

Parallelism is controlled by the number of partitions in RDDs; increasing partitions increases the level of parallelism.

103. What are the different input sources for Spark Streaming?

Input sources include:

- Kafka
- Flume
- Kinesis
- Socket
- HDFS or S3

104. Explain Spark Streaming with Socket.

Spark Streaming can receive real-time data streams over a socket using `socketTextStream()`.

105. Define the roles of the file system in any framework.

The file system manages data storage, access, and security, ensuring data integrity and availability.

106. How do you parse data in XML? Which kind of class do you use with Java to parse data?

To parse XML data in Java, you can use classes from the `javax.xml.parsers` package, such as:

- **DocumentBuilder**: Used with the Document Object Model (DOM) for in-memory tree representation.
- **SAXParser**: Used with the Simple API for XML (SAX) for event-driven parsing.

107. What is PageRank in Spark?

PageRank is an algorithm used to rank web pages in search engine results, based on the number and quality of links to a page. In Spark, it can be implemented using RDDs or DataFrames to compute the rank of nodes in a graph.

108. What are the roles and responsibilities of worker nodes in the Apache Spark cluster? Is the Worker Node in Spark the same as the Slave Node?

- **Worker Nodes**: Execute tasks assigned by the Spark Driver, manage executors, and store data in memory or disk as required.
- **Slave Nodes**: Worker nodes in Spark are commonly referred to as slave nodes. Both terms are used interchangeably.

109. How to split a single HDFS block into partitions in an RDD?

When reading from HDFS, Spark splits a single block into multiple partitions based on the number of available cores or executors. You can also use the `repartition()` method to explicitly specify the number of partitions.

110. On what basis can you differentiate RDD, DataFrame, and DataSet?

- **RDD**: Low-level, unstructured data; provides functional programming APIs.
- **DataFrame**: Higher-level abstraction with schema; optimized for SQL queries and transformations.
- **Dataset**: Combines features of RDDs and DataFrames; offers type safety and object-oriented programming.

Broadcast Hash Join

Suitable for scenarios where one of the datasets is small enough to be broadcasted to all executor nodes. The smaller dataset is broadcasted to all nodes in the cluster, and then each node joins the broadcasted dataset with partitions of the larger dataset locally. Efficient when one dataset is significantly smaller than the other.

Syntax:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import broadcast

spark =
SparkSession.builder.appName("JoinStrategies").getOrCreate()

smallDF = spark.read.csv("small_dataset.csv", header=True)
largeDF = spark.read.csv("large_dataset.csv", header=True)

joinedDF = largeDF.join(broadcast(smallDF), "key")
joinedDF.show()
```

Sort Merge Join

Default join strategy for large datasets that do not fit into memory. Both datasets are sorted by the join key and then merged. This requires a shuffle to sort the data, which can be expensive. Suitable for large datasets where both tables are too large to broadcast.

Syntax:

```
df1 = spark.read.csv("dataset1.csv", header=True)
df2 = spark.read.csv("dataset2.csv", header=True)
```

```
joinedDF = df1.join(df2, "key")
joinedDF.show()
```

Shuffle Hash Join

Used when the datasets are too large for a broadcast join, but one of the datasets is still relatively small. The smaller dataset is hashed and the data is partitioned based on the hash of the join keys. Each partition of the larger dataset is then joined with the corresponding partition of the smaller dataset. Efficient when one dataset is smaller but not small enough to broadcast.

Syntax:

```
df1 = spark.read.csv("large_dataset.csv", header=True)
df2 = spark.read.csv("medium_dataset.csv", header=True)
joinedDF = df1.hint("SHUFFLE_HASH").join(df2, "key")
joinedDF.show()
```

Broadcast Nested Loop Join

Used when no other join strategy is applicable, often as a fallback method. A Cartesian product of the datasets is produced, and then a filter is applied to select the matching rows. It broadcasts the smaller table and then iterates through it for each row of the larger table. Applicable in cross joins or when join conditions are complex and other joins are not feasible.

Synatx:

```
df1 = spark.read.csv("dataset1.csv", header=True)
df2 = spark.read.csv("dataset2.csv", header=True)
```

```
joinedDF = df1.crossJoin(df2)
joinedDF.show()
```

Cartesian Product Join

Computes the Cartesian product of two datasets. Every row of the first dataset is joined with every row of the second dataset. Used explicitly for cross joins or when the join condition is not specified.

Syntax:

```
df1 = spark.read.csv("dataset1.csv", header=True)
df2 = spark.read.csv("dataset2.csv", header=True)
joinedDF = df1.crossJoin(df2)
joinedDF.show()
```

Skew Join

Special strategy to handle data skew in join keys. Skewed keys are detected, and their corresponding partitions are split into smaller partitions to distribute the load evenly across the cluster. Useful when certain keys in the join have disproportionately large amounts of data.

Syntax:

```
spark.conf.set("spark.sql.auto.skewJoin.enabled", "true")
df1 = spark.read.csv("large_skewed_dataset.csv", header=True)
df2 = spark.read.csv("large_skewed_dataset.csv", header=True)
joinedDF = df1.join(df2, "key")
joinedDF.show()
```

Apache Spark

UPDATED BY TIM SPANN BIG DATA SOLUTIONS ENGINEER, HORTONWORKS
WRITTEN BY ASHWINI KUNTAMUKKALA SOFTWARE ARCHITECT, SCISPIKE

CONTENTS

- WHY APACHE SPARK?
- ABOUT APACHE SPARK
- HOW TO INSTALL APACHE SPARK
- HOW APACHE SPARK WORKS
- RESILIENT DISTRIBUTED DATASET
- DATAFRAMES
- RDD PERSISTENCE
- SPARK SQL
- SPARK STREAMING

DZONE.COM/REFCARDZ

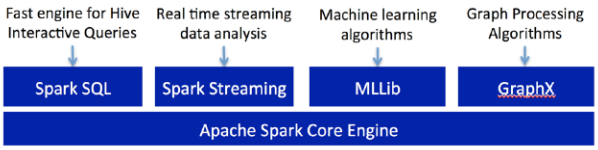
WHY APACHE SPARK?

Apache Spark has become the engine to enhance many of the capabilities of the ever-present Apache Hadoop environment. For Big Data, Apache Spark meets a lot of needs and runs natively on Apache Hadoop's YARN. By running Apache Spark in your Apache Hadoop environment, you gain all the security, governance, and scalability inherent to that platform. Apache Spark is also extremely well integrated with Apache Hive and gains access to all your Apache Hadoop tables utilizing integrated security.

Apache Spark has begun to really shine in the areas of streaming data processing and machine learning. With first-class support of Python as a development language, PySpark allows for data scientists, engineers and developers to develop and scale machine learning with ease. One of the features that has expanded this is the support for Apache Zeppelin notebooks to run Apache Spark jobs for exploration, data cleanup, and machine learning. Apache Spark also integrates with other important streaming tools in the Apache Hadoop space, namely Apache NiFi and Apache Kafka. I like to think of Apache Spark + Apache NiFi + Apache Kafka as the three amigos of Apache Big Data ingest and streaming. The latest version of Apache Spark is 2.2.

ABOUT APACHE SPARK

Apache Spark is an open source, Hadoop-compatible, fast and expressive cluster-computing data processing engine. It was created at AMPLabs in UC Berkeley as part of Berkeley Data Analytics Stack (BDAS). It is a top-level Apache project. The below figure shows the various components of the current Apache Spark stack.



It has six major benefits:

1. Lightning speed of computation because data are loaded in distributed memory (RAM) over a cluster of machines. Data can be quickly transformed iteratively and cached on demand for subsequent usage.
2. Highly accessible through standard APIs built in Java, Scala, Python, R, and SQL (for interactive queries) and has a rich set of machine learning libraries available out of the box.
3. Compatibility with existing Hadoop 2.x (YARN) ecosystems so companies can leverage their existing infrastructure.
4. Convenient download and installation processes. Convenient shell (REPL: Read-Eval-Print-Loop) to interactively learn the APIs.
5. Enhanced productivity due to high-level constructs that keep the focus on content of computation.
6. Multiple user notebook environments supported by Apache Zeppelin.

Also, Spark is implemented in Scala, which means that the code is very succinct and fast and requires JVM to run.

HOW TO INSTALL APACHE SPARK

The following table lists a few important links and prerequisites:

Current Release	2.2.0 @ apache.org/dyn/closer.lua/spark/spark-2.2.0/spark-2.2.0-bin-hadoop2.7.tgz
Downloads Page	spark.apache.org/downloads.html
JDK Version (Required)	1.8 or higher
Scala Version (Required)	2.11 or higher
Python (Optional)	[2.7, 3.5)
Simple Build Tool (Required)	scala-sbt.org
Development Version	github.com/apache/spark

Building Instructions	spark.apache.org/docs/latest/building-spark.html
Maven	3.3.9 or higher
Hadoop + Spark Installation	docs.hortonworks.com/HDPDocuments/Ambari-2.6.0.0/bk_ambari-i0nstallation/content/ch_Getting_Ready.html

Apache Spark can be configured to run standalone or on Hadoop 2 YARN. Apache Spark requires moderate skills in Java, Scala, or Python. Here we will see how to install and run Apache Spark in the standalone configuration.

1. Install JDK 1.8+, Scala 2.11+, Python 3.5+ and Apache Maven.
2. Download Apache Spark 2.2.0 Release.
3. Untar and unzip spark-2.2.0.tgz in a specified directory.
4. Go to the directory and run sbt to build Apache Spark.

```
export MAVEN_OPTS="-Xmx2g -XX:ReservedCodeCacheSize=512m"
mvn -Pyarn -Phadoop-2.7 -Dhadoop.version=2.7.3 -Phive
-Phive thriftserver -DskipTests clean package
```

5. Launch Apache Spark standalone REPL. For Scala, use:

```
./spark-shell
```

For Python, use:

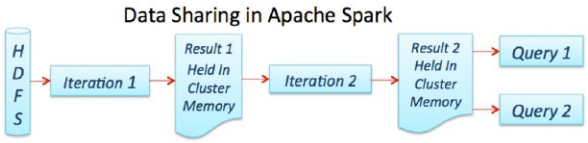
```
./pyspark
```

6. Go to SparkUI @ http://localhost:4040

This is a good quick start, but I recommend utilizing a Sandbox or an available Apache Zeppelin notebook to begin your exploration of Apache Spark.

HOW APACHE SPARK WORKS

The Apache Spark engine provides a way to process data in distributed memory over a cluster of machines. The figure below shows a logical diagram of how a typical Spark job processes information.



RESILIENT DISTRIBUTED DATASET

The core concept in Apache Spark is the resilient distributed dataset (RDD). It is an immutable distributed collection of data, which is partitioned across machines in a cluster. It facilitates two types of operations: transformations and actions. A transformation is an operation such as `filter()`, `map()`, or `union()` on an RDD that yields another RDD. An action is an operation such as `count()`, `first()`, `take(n)`, or `collect()` that triggers a computation, returns a value back to the Driver program, or writes to a stable storage system like Apache Hadoop HDFS. Transformations are lazily evaluated in that they don't run until an action warrants it. The Apache Spark Driver remembers the transformations applied to an RDD, so if a partition is lost (say a worker machine goes down), that partition can easily be reconstructed on some other machine in the cluster. That is why is it called "Resilient."

The following code snippets show how we can do this in Python using the Spark 2 PySpark shell.

```
%spark2.pyspark
guten = spark.read.text('/load/55973-0.txt')
```

COMMONLY USED TRANSFORMATIONS

Transformation & Purpose	Example & Result
`filter(func)` **Purpose:** new RDD by selecting those data elements on which `func` returns true	shinto = guten.filter(guten.Variable contains("Shinto"))

`map(func)`
Purpose: return new RDD by applying `func` on each data element
	`val rdd = sc.parallelize(List(1,2,3,4,5))`
	`val times2 = rdd.map(*2) times2. collect()`
	Result:
	`Array[Int] = Array(2, 4, 6, 8, 10)`

`flatMap(func)`
Purpose: Similar to `map` but `func` returns a sequence instead of a value. For example, mapping a sentence into a sequence of words
	`val rdd=sc. parallelize(List("Spark is awesome","It is fun"))`
	`val fm=rdd.flatMap(str=>str. split(" "))`
	`fm.collect()`
	Result:
	`Array[String] = Array(Spark, is, awesome, It, is, fun)`

`reduceByKey(-func,[numTasks])`
Purpose: To aggregate values of a key using a function. "numTasks" is an optional parameter to specify a number of reduce tasks
	`val word1=fm.map(word=>(word,1))`
	`val wrdCnt = word1. reduceByKey(_+_)wrdCnt.collect()`
	Result:
	`Array[(String, Int)] = Array((is,2), (It,1), (awesome,1), (Spark,1), (fun,1))`

`groupByKey([num-Tasks])`
Purpose: To convert (K,V) to (K,Iterable<V>)
	`val cntWrd = wrdCnt.map{case (word, count) => (count, word)}`
	`cntWrd.groupByKey().collect()`
	Result:
	`Array[(Int, Iterable[String])] = Array((1,ArrayBuffer(It, awesome, Spark, fun)), (2,ArrayBuffer(is)))`

DZONE.COM/REFCARDZ

`distinct([numTasks])`
Purpose: Eliminate duplicates from RDD
	`fm.distinct().collect()`
	Result:
	`Array[String] = Array(is, It, awesome, Spark, fun)`

COMMONLY USED SET OPERATIONS

Transformation & Purpose	Example & Result

`filter(func)`
Purpose: new RDD by selecting those data elements on which `func` returns true
	`val rdd1=sc.parallelize(List('A','B'))`
	`val rdd2=sc.parallelize(List('B','C'))`
	`rdd1.union(rdd2).collect()`
	Result:
	`Array[Char] = Array(A, B, B, C)`

`cartesian()`
Purpose: new RDD cross product of all elements from source RDD and argument
	`rdd1.cartesian(rdd2).collect()`
	Result:
	`Array[(Char, Char)] = Array((A,B), (A,C), (B,B), (B,C))`

`subtract()`
Purpose: new RDD created by removing data elements in source RDD in common with argument
	`rdd1.subtract(rdd2).collect()`
	Result:
	`Array[Char] = Array(A)`

`join(RDD,[numTasks])`
Purpose: When invoked on (K,V) and (K,W), this operation creates a new RDD of (K, (V,W))
	`val personFruit = sc.parallelize(Seq((“Andy”, “Apple”), (“Bob”, “Banana”), (“Charlie”, “Cherry”), (“Andy”,”Apricot”)))`
	`val personSE = sc.parallelize(Seq((“Andy”, “Google”), (“Bob”, “Bing”), (“Charlie”, “Yahoo”), (“Bob”,”AltaVista”)))`
	`personFruit.join(personSE).collect()`
	Result:
	`Array[(String, (String, String))] = Array((Andy,(Apple,Google)), (Andy,(Apricot,Google)), (Charlie,(Cherry,Yahoo)), (Bob,(Banana,Bing)), (Bob,(Banana,AltaVista)))`

cogroup(RDD,[numTasks])
Purpose: To convert (K,V) to (K,Iterable<V>)
	`personFruit.cogroup(personSe).collect()`
	Result:
	`Array[(String, (Iterable[String], Iterable[String]))] = Array((Andy,(ArrayBuffer(Apple, Apricot),ArrayBuffer(google))), (Charlie,(ArrayBuffer(Cherry), ArrayBuffer(Yahoo))), (Bob,(ArrayBuffer(Banana), ArrayBuffer(Bing, AltaVista))))`

For a more detailed list of transformations, please refer to spark.apache.org/docs/latest/programming-guide.html#transformations.

COMMONLY USED ACTIONS

Action & Purpose	Example & Result

`count()`
Purpose: get the number of data elements in the RDD
	`val rdd = sc.parallelize (list('A','B','c')) scala> rdd.count()`
	Result:
	`long = 3`

`collect()`
Purpose: get all the data elements in an RDD as an array
	`val rdd = sc.parallelize(list ('A','B','c'))rdd.collect()`
	Result:
	`Array[char] = Array(A, B, c)`

`reduce(func)`
Purpose: Aggregate the data elements in an RDD using this function which takes two arguments and returns one
	`val rdd = sc.parallelize(list(1,2,3,4))`
	`rdd.reduce(_+_)`
	Result:
	`Int = 10`

`take (n)`
Purpose: fetch first n data elements in an RDD, computed by driver program
	`val rdd = sc.parallelize(list(1,2,3,4))`
	`rdd.take(2)`
	Result:
	`Array[Int] = Array(1, 2)`

DZONE.COM/REFCARDZ

`foreach(func)`
Purpose: execute function for each data element in RDD, usually used to update an accumulator (discussed later) or interacting with external systems
	`val rdd = sc.parallelize(list(1,2,3,4)) rdd. foreach(x=>println("%s*10=%s". format(x,x*10)))`
	Result:
	`1*10=10 4*10=40 3*10=30 2*10=20`

`first()`
Purpose: retrieves the first data element in RDD. Similar to take(1)
	`val rdd = sc.parallelize(list(1,2,3,4)) rdd.first()`
	Result:
	`Int = 1`

`saveAsTextFile(path)`
Purpose: Writes the content of RDD to a text file or a set of text files to local file system/ HDFS
	`val hamlet = sc.textFile("/users/akuntamukkala/ temp/gutenburg.txt") hamlet.filter(_.contains("Shakespeare")). saveAsTextFile("/users/akuntamukkala/temp/ filtered")`
	Result:
	`akuntamukkala@localhost~/temp/filtered$ ls _SUCCESS part-00000 part-00001`

For a more detailed list of actions, please refer to spark.apache.org/docs/latest/programming-guide.html#actions.

RDD PERSISTENCE

One of the key capabilities in Apache Spark is persisting/caching RDD in cluster memory. This speeds up iterative computation.

The following table shows the various options Spark provides:

Storage Level	Purpose
`MEMORY_ONLY (Default level)`	This option stores RDD in available cluster memory as deserialized Java objects. Some partitions may not be cached if there is not enough cluster memory. Those partitions will be recalculated on the fly as needed.
`MEMORY_AND_DISK`	This option stores RDD as deserialized Java objects. If RDD does not fit in cluster memory, then store those partitions on the disk and read them as needed.
`MEMORY_ONLY_SER`	This options stores RDD as serialized Java objects (One byte array per partition). This is more CPU intensive but saves memory as it is more space efficient. Some partitions may not be cached. Those will be recalculated on the fly as needed.
`MEMORY_ONLY_DISK_SER`	This option is same as above except that disk is used when memory is not sufficient.
`DISK_ONLY`	This option stores the RDD only on the disk
`MEMORY_ONLY_2, MEMORY_AND_DISK_2, etc.`	Same as other levels but partitions are replicated on 2 slave nodes
`OFF_HEAP (experimental)`	Works off of JVM heap and must be enabled.

The above storage levels can be accessed through persist() operation on RDD. The `cache()` operation is a convenient way of specifying a `MEMORY_ONLY` option. The **SER** options do not work with Python.

For a more detailed list of persistence options, please refer to http://spark.apache.org/docs/latest/programming-guide.html#rdd-persistence

Spark uses the Least Recently Used (LRU) algorithm to remove old, unused cached RDDs to reclaim memory. It also provides a convenient `unpersist()` operation to force removal of cached/persisted RDDs.

DATAFRAMES

Action & Purpose	Example & Result

`printSchema()`
Purpose: print out the schema of the DataFrame
	`df.printSchema()`
	Result:
	`root |-- clientIp: string (nullable = true) |-- clientIdentity: string (nullable = true) |-- user: string (nullable = true) |-- dateTime: string (nullable = true) |-- request: string (nullable = true) |-- statusCode: integer (nullable = true) |-- bytesSent: long (nullable = true) |-- referer: string (nullable = true) |-- userAgent: string (nullable = true)`

`collect()`
Purpose: returns all the records as a list of rows
	`df.collect()`
	Result:
	`Row(value=u'The Project Gutenberg EBook of Shinto: The ancient religion of Japan, by ')`

`columns()`
 	
Purpose: returns all the columns as a list
	`df.columns()`
	
	Result:
	`['userAgent', 'referer', 'bytesSent']`

`count()`
 	
Purpose: returns number of rows
	`df.count()`
	
	Result:
	`2`

`createTempView()` **Purpose:** create a local temporary view that can be used in Spark SQL	`df.createTempView("viewName")`

A DataFrame is a distributed collection of data with named columns built on the Dataset interface. You can learn more here: spark.apache.org/docs/latest/sql-programming-guide.html.

SHARED VARIABLES

ACCUMULATORS

Accumulators are variables that can be incremented in distributed tasks.

```
exampleAccumulator = sparkContext.accumulator(1)
exampleAccumulator.add(5)
```

BROADCAST VARIABLES

Using the SparkContext, you can broadcast a read-only value to other tasks. You can set, destroy, and unpersist these values.

```
broadcastVariable = sparkContext.broadcast(500)
broadcastVariable.value
```

SPARK SQL

Spark SQL provides a convenient way to run interactive queries over large data sets using Apache Spark Engine, returning DataFrames. Spark SQL provides two types of contexts, SQLContext and HiveContext, that extend SparkContext functionality.

SQLContext provides access to a simple SQL parser, whereas HiveContext provides access to the HiveQL parser. HiveContext enables enterprises to leverage their existing Hive infrastructure.

Let's see a simple example in Scala:

```
val df = spark.read.csv("customers.txt")
val dfS = spark.sql("select * from customers where gender='M'")
dfs.printSchema()
dfs.show()
```

Here's one in Python for Apache Hive:

```
spark = SparkSession.builder.appName("dzone1").config("spark.sql.
warehouse.dir", "/mydata").enableHiveSupport().getOrCreate()
spark.sql("SELECT * FROM default.myHiveTable")
```

For more practical examples using SQL & HiveQL, please refer to the following link: spark.apache.org/docs/latest/sql-programming-guide.html.

SPARK STREAMING

Spark Streaming provides a scalable, fault tolerant, efficient way of processing streaming data using Spark's simple programming model. It converts streaming data into "micro" batches, which enable Spark's batch programming model to be applied in Streaming use cases. This unified programming model makes it easy to combine batch and interactive data processing with streaming.

The core abstraction in Spark Streaming is Discretized Stream (DStream). DStream is a sequence of RDDs. Each RDD contains data received in a configurable interval of time.

Spark Streaming also provides sophisticated window operators, which help with running efficient computation on a collection of RDDs in a rolling window of time. DStream exposes an API, which contains operators (transformations and output operators) that are applied on constituent RDDs. Let's try and understand this using a simple example:

```
import org.apache.spark._
import org.apache.spark.streaming._
val conf = new SparkConf().setAppName("appName").
setMaster("masterNode")
val ssc = new StreamingContext(conf, Seconds(1))
val lines = ssc.socketTextStream("localhost", 9999)
```

The above snippet is setting up Spark Streaming Context. Spark Streaming will create an RDD in DStream containing text network streams retrieved every second.

There are many commonly used source data streams for Spark Streaming, including Apache Kafka, Apache HDFS, Twitter, Apache NiFi S2S, Amazon S3, and Amazon Kinesis.

Transformation & Purpose	Example & Result
`map(func)` **Purpose:** Create new DStream by applying this function to tall constituent RDDS in DStream	`lines.map(x=>x.tolnt*10).print()` nc –lk 9999 12 34 Output: 120 340

`flatMap(func)` **Purpose:** This is the same as `map` but mapping function can output 0 or more items	`lines.flatMap(_.split(" "))).print()` nc –lk 9999 Spark is fun Output: Spark is fun
`count()` **Purpose:** create a DStream of RDDs containing a count of the number of data elements	`lines.flatMap(_.split(" "))).print()` nc –lk 9999 say hello to spark Output: 4
`reduce(func)` **Purpose:** Same as `count` but the value is derived by applying the function	`lines.map(x=>x.toInt).reduce(_+_).print()` nc –lk 9999 1 3 5 7 Output: 16
`countByValue()` **Purpose:** This is same as `map` but mapping function can output 0 or more items	`lines.map(x=>x.toInt).reduce(_+_).print()` nc –lk 9999 spark spark is fun fun Output: (is,1) (spark,2) (fun,2)
`reduceByKey (func,[numTasks])`	`lines.map(x=>x.toInt).reduce(_+_).print()` nc –lk 9999 spark spark is fun fun Output: (is,1) (spark,2) (fun,2)
`reduceByKey (func,[numTasks])`	val words = lines.flatMap(_.split(" ")) val wordCounts = words.map(x => (x, 1)). reduceByKey(_+_) wordCounts.print() nc –lk 9999 spark is fun fun fun Output: (is,1) (spark,1) (fun,3)

The following example shows how Apache Spark combines Spark batch with Spark Streaming. This is a powerful capability for an all-in-one technology stack. In this example, we read a file containing brand names and filter those streaming data sets that contain any of the brand names in the file.

`transform(func)`
 	
Purpose: Creates a new DStream by applying RDD->RDD transformation to all RDDs in DStream
 	
brandNames.txt
coke
nike
sprite
reebok
	`val sc = new SparkContext(sparkConf)`
	`val ssc = new StreamingContext(sc, Seconds(10))`
	`val lines = ssc. socketTextStream("localhost" 9999, StorageLevel.MEMORY_AND_DISK_SER)`
	`val brands = sc.textFile("/tmp/names.txt")`
	`lines.transform(rdd=> { rdd.intersection(brands) }).print()`
	
	nc –lk 9999
	msft
	apple
	nike
	sprite
	ibm
	
	Output:
	sprite
	nike

COMMON WINDOW OPERATIONS

Transformation & Purpose	Example & Result
`window(windowLength, slideInterval)`	`val win = lines. window(Seconds(30),Seconds(10)); win. foreachRDD(rdd => { rdd.foreach(x=>println(x+ " ")) })` nc –lk 9999 10 (0th second) 20 (10 seconds later) 30 (20 seconds later) 40 (30 seconds later) Output: 10 10 20 20 10 30 20 30 40 (drops 10)
`countByWindow (windowLength slideInterval)` **Purpose:** Returns a new sliding window count of elements in a steam	lines.countByWindow(Seconds(30), Seconds (10)).print() nc –lk 9999 10 (0th second) 20 (10 seconds later) 30 (20 seconds later) 40 (30 seconds later) Output: 1 2 3 3

For additional transformation operators, please refer to spark.apache.org/docs/latest/streaming-programming-guide.html#transformations.

DZONE.COM/REFCARDZ

Spark Streaming has powerful output operators. We already saw foreachRDD() in the above example. For others, please refer to spark.apache.org/docs/latest/streaming-programming-guide.html#output-operations.

Structured Streaming has been added to Apache Spark and allows for continuous incremental execution of a structured query. There a few input sources supported including files, Apache Kafka, and sockets. Structured Streaming supports windowing and other advanced streaming features. It is recommended when streaming from files that you supply a schema as opposed to letting Apache Spark infer one for you. This is a similar feature of most streaming systems, like Apache NiFi and Hortonworks Streaming Analytics Manager.

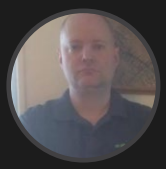
```
val sStream = spark.readStream.json("myJson").load()
sStream.isStreaming

sStream.printSchema
```

For more details on Structured Streaming, please refer to spark.apache.org/docs/latest/structured-streaming-programming-guide.html.

ADDITIONAL RESOURCES

- docs.hortonworks.com/HDPDocuments/HDP2/HDP-2.6.3/bk_spark-component-guide/content/ch_introduction-spark.html
- hortonworks.com/tutorial/a-lap-around-apache-spark
- hortonworks.com/products/sandbox
- hortonworks.com/tutorial/hands-on-tour-of-apache-spark-in-5-minutes
- hortonworks.com/tutorial/sentiment-analysis-with-apache-spark
- spark.apache.org/downloads.html
- spark.apache.org/docs/latest
- spark.apache.org/docs/latest/quick-start.html
- zeppelin.apache.org
- jaceklaskowski.gitbooks.io/mastering-apache-spark



Written by Tim Spann

is a Big Data Solution Engineer. He helps educate and disseminate performant open source solutions for Big Data initiatives to customers and the community. With over 15 years of experience in various technical leadership, architecture, sales engineering, and development roles, he is well-experienced in all facets of Big Data, cloud, IoT, and microservices. As part of his community efforts, he also runs the Future of Data Meetup in Princeton.

DZone

DZone communities deliver over 6 million pages each month to more than 3.3 million software developers, architects and decision makers. DZone offers something for everyone, including news, tutorials, cheat sheets, research guides, feature articles, source code and more. "DZone is a developer's dream," says PC Magazine.

DZone, Inc.
150 Preston Executive Dr. Cary, NC 27513
888.678.0399 919.678.0300

Copyright © 2017 DZone, Inc. All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by means electronic, mechanical, photocopying, or otherwise, without prior written permission of the publisher.

Code Optimization in PySpark: Best Practices for High Performance

Apache Spark is a powerful framework for distributed data processing, but to fully leverage its capabilities, it's essential to write efficient PySpark code. Optimizing your Spark code can lead to significant improvements in performance and resource utilization. In this blog post, we'll explore various techniques and best practices for optimizing PySpark code.

Understanding Spark's Lazy Evaluation

One of the core concepts in Spark is lazy evaluation. Transformations on RDDs, DataFrames, or Datasets are not executed immediately. Instead, they are recorded as a lineage of operations to be applied when an action is called. This lazy evaluation allows Spark to optimize the execution plan.

Best Practice:

- **Minimize the number of transformations**: Chain transformations together and avoid unnecessary intermediate operations.
- **Use actions wisely**: Trigger actions (like `collect()`, `count()`, etc.) only when necessary.

Use the DataFrame API Over RDDs

DataFrames provide a higher-level abstraction than RDDs and come with a Catalyst optimizer that can automatically optimize queries.

Best Practice:

- **Prefer DataFrames over RDDs**: Use the DataFrame API for better performance and easier code.
- **Leverage SQL queries**: Use SQL for complex transformations, taking advantage of Spark's Catalyst optimizer.

Caching and Persistence

Caching and persisting DataFrames or RDDs can improve performance, especially for iterative algorithms or when the same data is accessed multiple times.

Best Practice:

- **Cache DataFrames/RDDs**: Use `df.cache()` or `df.persist()` to store frequently accessed data in memory.

- **Choose the right storage level**: Use appropriate storage levels (e.g., MEMORY_ONLY, MEMORY_AND_DISK) based on your application's needs.

```
# Example of caching a DataFrame
df = spark.read.csv("data.csv")
df.cache()
```

Partitioning and Coalescing

Efficient data partitioning can significantly impact performance. Proper partitioning reduces shuffling and improves data locality.

Best Practice:

- **Repartition DataFrames**: Use `df.repartition(n)` to increase or decrease the number of partitions.
- **Coalesce DataFrames**: Use `df.coalesce(n)` to reduce the number of partitions without full shuffle.

```
# Example of repartitioning a DataFrame
df = df.repartition(10)
```

Avoid UDFs (User-Defined Functions) When Possible

While UDFs provide flexibility, they can be slow because they prevent Spark from optimizing the execution plan.

Best Practice:

- **Use built-in functions**: Leverage Spark's built-in functions (`pyspark.sql.functions`) instead of UDFs for better performance.
- **Pandas UDFs**: If UDFs are necessary, use Pandas UDFs which can be more efficient.

```
# Example using a built-in function
from pyspark.sql.functions import col, sqrt
df = df.withColumn("sqrt_col", sqrt(col("value")))
```

Broadcast Joins

For small datasets, broadcasting can be more efficient than a standard join, as it avoids shuffling the larger dataset.

Best Practice:

- **Broadcast small DataFrames**: Use `broadcast()` for small lookup tables.

```
# Example of a broadcast join
from pyspark.sql.functions import broadcast

small_df = spark.read.csv("small_data.csv")
large_df = spark.read.csv("large_data.csv")

joined_df = large_df.join(broadcast(small_df), "key")
```

Use Window Functions Wisely

Window functions are powerful for performing operations over a specified window of rows, but they can be expensive.

Best Practice:

- **Optimize window functions**: Use partitioning within window functions to minimize the data processed.

```
# Example of a window function
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number

window_spec = Window.partitionBy("category").orderBy("value")
df = df.withColumn("row_number", row_number().over(window_spec))
```

Reduce Data Shuffling

Shuffling data across the network is expensive. Minimize shuffles by using techniques such as partitioning, avoiding wide transformations when possible, and careful join strategies.

Best Practice:

- **Optimize joins**: Use broadcast joins for small tables and avoid joining large datasets unnecessarily.
- **ReduceByKey over GroupByKey**: Use `reduceByKey` instead of `groupByKey` to minimize the amount of data shuffled.

Monitor and Tune Spark Configurations

Proper Spark configuration tuning can significantly impact performance. Monitor your Spark application using Spark UI and adjust configurations as needed.

Best Practice:

- **Tune executor memory and cores**: Configure `spark.executor.memory` and `spark.executor.cores` based on your cluster resources and application requirements.
- **Adjust shuffle partitions**: Set `spark.sql.shuffle.partitions` to an appropriate number based on the data size and cluster capacity.

Write Efficient Code

Efficient code not only runs faster but is also easier to read and maintain. Follow best coding practices to write clean, efficient PySpark code.

Best Practice:

- **Use vectorized operations**: Leverage vectorized operations in DataFrames for better performance.
- **Avoid using collect() on large datasets**: Use `collect()` only on small datasets to avoid driver memory overload.